

DSL for querying RDF turtle format documents

January 9, 2021

Revised 4 September 2026 - see section 10, Changes since submission.

Ishaipiriyam Karunakularatnam

ik3g20@soton.ac.uk

Danny M Hibbert

dmh1g19@soton.ac.uk

Robert Stoica

rss1g20@soton.ac.uk

Contents

- 1 Introduction
- 2 Language Design
- 3 Syntax
 - 3.1 Program semantics
 - 3.2 Scope
- 4 Functions
 - 4.1 IMPORT
 - 4.2 AS
 - 4.3 INTO
 - 4.4 GET
 - 4.5 WHERE
 - 4.6 EXPORT
 - 4.7 LISTS
 - 4.8 Conditionals
 - 4.9 FROM
- 5 Grammar
- 6 Example code
 - 6.1 Linking from different files
 - 6.2 Writen output using conditionals
 - 6.3 Pattern matching triple conditions
- 7 Type checking and error handling
- 8 Language review
- 9 Building and testing
- 10 Changes since submission

1 Introduction

This report discusses the design and implementation of a domain specific programming language for querying RDF documents in turtle format with bases and prefixes. The language is written in Haskell using Alex for tokenizing and Happy for parsing.

2 Language Design

This language stands out predominantly from other general query languages in that it only deals with turtle formats strictly containing triples of the form `<subject>`

`<predicate> <object>`. The design of the language closely relates to standard RDF querying languages with a simplified and straight forward approach, literals are always expected to be of an integer format.

The language uses a set of keywords to filter turtle files after specifying importing instructions and accompanying variable assignments, type checking is performed through the use of the program structure - not explicit data for type scoping is defined.

3 Syntax

The syntax of this language is supposed to be a very simplified take on SQL for turtle languages that only queries/edits and returns turtle RDF formatted text. Functions are defined in capital letters and are pre-defined by the language and cannot be overloaded. File names can be assigned a user defined variable, and any filters can be provided within **WHERE** blocks.

3.1 Program semantics

The semantics of the program follows Big Step operational semantics. We parse the program bottom up, which is then reversed before evaluation each step of the program one by one while building up to the output. A relationship is maintained between each term and their associated values during run time to output a final result. As part of the interpreter we implemented a simple CEK machine through the use of an Environment, Kontinuation and Frame list to evaluate each step of the program. We used the environment to store the values and Kontinuation to make sure the program follows our defined syntax. We focused on big step operational semantics as our program heavily focuses on producing specific values as outputs.

3.2 Scope

Due to the implementation of the language structure and the method used for tokenizing each line - scope of variables are kept globally accessible before exporting the final output.

4 Functions

4.1 IMPORT

The language scopes **.ttl** files using the **IMPORT** function. The function is called before retrieval of triples and any application of filters.

The file name is given without its extension - **IMPORT foo** reads **foo.ttl**.

```
IMPORT foo...
```

4.2 AS

AS provides a reference assignment **Var** for using in filtering processes within **WHERE** blocks.

```
IMPORT foo AS A...
```

4.3 INTO

The **.ttl** file to write to is defined at the start of the main block of code using the **INTO** function.

```
INTO out1...
```

4.4 GET

The triple format is specified as the data to retrieve from the imported **.ttl** file through the use of **GET** followed by a list.

```
GET [subj,pred,obj]...
```

4.5 WHERE

The **WHERE** block is followed by a section in curly brackets containing all filters to be applied to the triple defined in the **GET** block.

```
WHERE A[subj,pred,obj]...
```

4.6 EXPORT

Outputting the final written **.ttl** file can be done after the main code block of the program using the **EXPORT** function followed by the filename defined earlier in the program using the **INTO** function. **EXPORT out1** writes **out1.ttl** and also echoes its contents on standard output.

```
EXPORT out1...
```

4.7 LISTS

Lists are depicted within square brackets and can contain element definitions in regards to turtle style triples and other literal values such as bools. Lists can represent the contents of different files with suffixes.

```
WHERE [subj,pred,obj]...  
WHERE B[subj,pred IN A,obj]...  
WHERE A[subj IN B,pred IN C,obj IN D]...
```

4.8 Conditionals

Conditionals are used to filter through more complex conditions. A condition compares a triple position against an integer using `<`, `>`, `<=`, `>=`, `=` or `!=`, and conditions combine with **AND**, **OR** and **NOT**.

```
IF {A[obj >= 0 AND obj <= 99]} THEN...  
IF {A[obj != 50]} THEN...  
IF {A[NOT obj > 10 AND obj != 1]} THEN...
```

NOT binds tighter than **AND** and **OR**, so `NOT obj > 10 AND obj != 1` reads as `(NOT (obj > 10)) AND (obj != 1)`.

Ordering comparisons are numeric and apply only to integer objects - a URI, boolean or string literal satisfies none of `<`, `>`, `<=`, `>=`. Equality behaves differently: a non-integer object is *not* equal to any integer, so it satisfies `!=` and fails `=`. This keeps **obj = n** and **obj != n** a partition of the data.

An **IF** condition is evaluated per line. **WRITETRUE** writes the lines that matched and **WRITEFALSE** those that did not, so both branches of an **IF** run. An **ELSE** branch only sees the lines its own **IF** condition did not match, so a chain of **ELSE IF** assigns each line to exactly one branch.

4.9 FROM

FROM is the shorthand for retrieving every triple of a variable with no filtering. These two statements are equivalent.

```
INTO out1 GET [subj,pred,obj] FROM A;  
INTO out1 GET [subj,pred,obj] WHERE {A[subj,pred,obj]};
```

5 Grammar

```
<prog> ::= <stmt>
| <prog> <stmt>

<stmt> ::= <exp>;

<exp> ::= <int>
| <var>
| NOTHING
| <function>
| <int> <operator> <int>
| <ifStatement>
| (<exp>)

<function> ::= IMPORT <exp> AS <exp>
| INTO <exp> <exp>
| GET [<list>] WHERE {<listCompare>}
| GET [<list>] FROM <exp>
| WRITE {<listCompare>}
| WRITETRUE {<listCompare>}
| WRITEFALSE {<listCompare>}
| IN <exp>
| AS <exp>
| EXPORT <exp>

<operator> ::= < | > | + | - | <= | >=

<ifStatement> ::= IF {<conditions>} THEN <exp> ELSE <exp>

<conditions> ::= <exp> [<condition>]
| <exp> [<condition>] OR <conditions>
| <exp> [<condition>] AND <conditions>

<condition> ::= <condStatement>
| <condStatement> OR <condition>
| <condStatement> AND <condition>

<condStatement> ::= <bool>
| <triple> <comparator> <int>
| NOT <condStatement>
| <exp>

<comparator> ::= < | > | <= | >= | = | !=

<list> ::= <listContent>
| <listContent>, <list>

<listContent> ::= <triple>
| <bool>
| <exp>

<triple> ::= subj | pred | obj
| subj IN <var> | pred IN <var> | obj IN <var>
| subj + <int> | pred + <int> | obj + <int>
| subj - <int> | pred - <int> | obj - <int>
```

```
<listCompare> ::= <exp>[<list>]  
| <exp>[<list>] <comparison> <listCompare>  
  
<comparison> ::= OR | AND  
  
<bool> ::= true | false  
  
<int> ::= [0-9]+  
  
<var> ::= [a-zA-Z][a-zA-Z0-9:_'. $|*?#~^/]*
```

6 Example code

6.1 Linking from different files

```
IMPORT foo AS A;  
IMPORT bar AS B;  
  
INTO out4 GET [subj, pred, obj] WHERE {A[subj,pred,subj IN B] AND B[subj,pred,subj  
IN A]};  
  
EXPORT out4;
```

6.2 Writen output using conditionals

```
IMPORT foo AS A;  
  
IF {A[obj < 0 OR obj > 99]} THEN  
INTO out5 WRITETRUE {A[subj, http://www.cw.org/problem5/#inRange, false]}  
ELSE IF {A[obj >= 0 AND obj <= 99]} THEN  
INTO out5 WRITETRUE {A[subj, pred, obj + 1] AND A[subj,  
http://www.cw.org/problem5/#inRange, true]}  
ELSE NOTHING;  
EXPORT out5;
```

6.3 Pattern matching triple conditions

A triple position can be matched against a literal, and alternatives are combined with **OR** inside the **WHERE** block. This selects every triple whose predicate is one of three given values.

```

IMPORT foo AS A;

INTO out3 GET [subj,pred,obj] WHERE
{A[subj,http://www.cw.org/problem3/#predicate1,obj]
  OR A[subj,http://www.cw.org/problem3/#predicate2,obj]
  OR A[subj,http://www.cw.org/problem3/#predicate3,obj]};

EXPORT out3;

```

7 Type checking and error handling

The language is designed to be as simple as possible and perform any type checking sequentially during the interpretation. The program will catch and throw errors in regards to input that does not adhere to a passable AST.

8 Language review

The implementation of the language lacks appropriate error checking methods, flexibility and misses some crucial implementations to solve more complex problems. There are areas to be improved with the language such as union operations as well as the inclusion of a unary minus operator to accommodate for negative numbers.

9 Building and testing

The build needs **ghc**, **alex** and **happy**. The lexer and parser are generated from **Tokens.x** and **Grammar.y**; **Grammar.hs** and **Tokens.hs** are checked in so the project also builds without Alex and Happy present.

```

make          # generates the lexer and parser, then builds ./stql
make test     # runs tests/prN.stql and diffs the file each one exports
make clean
./stql tests/pr1.stql

```

make report regenerates **rdf.pdf** from this README, so the report and the code cannot drift apart again. It needs **python3-markdown** and **google-chrome**; the other targets do not.

The **tests** directory holds the input turtle files, the example programs **pr1.stql** to **pr9.stql**, and their expected output under **tests/expected**. A program is run in a scratch directory and the file its **EXPORT** writes is compared against the recorded expectation. Failures are reported per test and **make test** exits non-zero.

10 Changes since submission

The report above describes the language as designed. The following corrections were made to the implementation afterwards, and the behaviour documented in sections 4 and 5 reflects the fixed version.

Build and tooling

- **eval.hs** was renamed **Eval.hs**. The module is named **Eval** and **Stql.hs** imports it, so the project only built on a case insensitive filesystem.
- A layout error in **eval** meant the project did not compile at all.
- A makefile and the regression suite described in section 9 were added.

Lexer and parser

- **(** and **)** were bound to each other's tokens, so **(exp)** was rejected and **)exp(** accepted.
- **{** and **}** were members of the identifier character class, so a keyword or variable written next to a brace was swallowed into one token - **IF{** lexed as a single variable.
- **tokenPosn** was missing four constructors, so a parse error at **AS**, **WHERE**, **FROM** or **IF** crashed the error reporter instead of reporting a position.
- **FROM**, **NOT** and **!=** were produced by the lexer but absent from the grammar. They are now accepted, with the meanings given in sections 4.8 and 4.9.
- An undocumented **IMPORT ... AS ... {var}** rule that no evaluator clause handled was removed.
- **IN** took an arbitrary expression, which made `subj IN 5 < 3` ambiguous - it could parse as `(subj IN 5) < 3` or as `subj IN (5 < 3)`. Every evaluator clause only ever handled a variable there, so **IN** now takes a variable and the four remaining shift/reduce conflicts are gone. Writing anything else after **IN** was previously accepted and silently ignored; it is now a parse error. The grammar's precedence declarations were vestigial - one of them named a symbol that did not exist - and were removed.

Evaluation

- **WRITE**, **WRITETRUE** and **WRITEFALSE** built their result in the list monad, so only the first matching line of a file was ever written.
- Comparisons between a triple position and an integer compared strings, so **obj > 10** matched the object **5**, and non-numeric objects satisfied every ordering comparison.
- **GET** inside an **IF** branch crashed, and a nested **ELSE IF** was silently discarded, because branches were stepped by a function that handled only a few expression forms.
- An **ELSE** branch re-tested every line rather than only those its **IF** did not match, so a chain of **ELSE IF** could assign one line to several branches.

- **INTO** reset its target on entry, so two branches writing to the same file lost the first branch's output.
- Arithmetic such as **obj + 1** in a write list produced a malformed triple.
- Prefixed names were corrupted inside predicate and object lists, and only single character prefix names were recognised.
- Multi-variable **IF** conditions crashed.

Error handling

- Failures were caught only for one exception type, so a missing file or a failed pattern match escaped uncaught, and a parse error still exited with status zero. Every failure is now reported on standard error with a non-zero exit status.